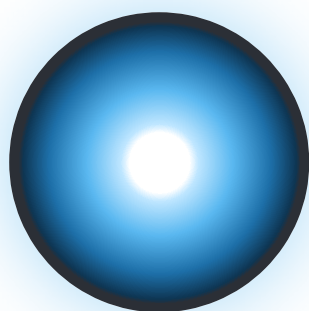




PLAN JARVIS

Osobisty asystent AI w stylu Jarvisa z Iron Mana,
połączony z Telegramem i działający na Windowsie,
z mózgiem na Ollamie: lokalnie i za darmo



Ollama · qwen2.5:7b



Telegram



Whisper



Piper TTS



Windows



Claude (opcjonalnie)

Wersja 2.0 (Ollama) · październik 2026 · github.com/MLG44-skr/jarvis

🎯 Założenia budżetowe

Element	Rozwiązanie	Koszt
🧠 Mózg	Ollama + <code>qwen2.5:7b</code> lokalnie na Twoim PC	0 zł
🌟 Mózg „na trudne sprawy”	<i>Opcjonalnie</i> Claude Haiku 4.5 (\$1 / \$5 za 1 mln tokenów), odpalany tylko na żądanie komendą <code>/madry</code>	0-10 zł/mies.
👂 Słuch	Whisper lokalnie (<code>faster-whisper</code>)	0 zł
🗣️ Głos	Piper TTS lokalnie (polskie głosy)	0 zł
💬 Interfejs	Telegram Bot API, czyli apka na telefon i komputer za darmo	0 zł
💻 Hosting	Twój komputer z Windowsem	0 zł

Łącznie: 0 zł miesięcznie (z opcjonalnym Claude kilka zł, tylko gdy go użyjesz)

⚠️ Ważne: API Claude to osobny rachunek, niezależny od subskrypcji Claude Pro/Max. Jest całkowicie opcjonalne, bo Jarvis działa w pełni na samej Ollamie.

Ollama vs Claude: uczciwie

	Ollama (qwen2.5:7b)	Claude Haiku 4.5
Koszt	0 zł	grosze za rozmowę
Prywatność	100%, nic nie wychodzi z PC	tekst idzie do API
Inteligencja	OK do gadki, notatek, przypomnień	dużo bystrzejszy
Polski	dobry, czasem wtrąca angielski/chiński	bardzo dobry
Bez internetu	✅ (poza samym Telegramem)	❌



Architektura

Ty (Telegram: tekst / głosówka)



Bot w Pythonie (na Twoim PC z Windowsem)

- | głosówka → ffmpeg → Whisper → tekst
- | tekst + pamięć + narzędzia → MÓZG
 - | domyślnie: Ollama (qwen2.5:7b) @ localhost:11434
 - | /madry: Claude Haiku (opcjonalnie)
- | odpowiedź → tekst (+ opcjonalnie Piper → głosówka)
- | SQLite: pamięć, notatki, przypomnienia

Mózg jest **wymienny**: jedna zmienna w `.env` (`BRAIN=ollama` albo `BRAIN=claude`), a reszta kodu się nie zmienia. W przyszłości można podmienić model w Ollamie (np. na nowszego Qwena albo polskiego **Bielika**) bez ruszania kodu.



Plan działania

Etap 0 Przygotowanie

~20 min · 0 zł

1. **Ollama**: masz zainstalowaną.
2. **Model** `qwen2.5:7b`: masz pobrany. Sprawdzenie: `ollama list` w PowerShellu.
3. **Bot Telegram**: napisz do **@BotFather**, wpisz `/newbot` i zapisz **token bota**.
4. **Twoje Telegram ID**: napisz do **@userinfobot** i zapisz swoje ID (do whitelisty).
5. **Python na Windowsie**: Python 3.11+ z python.org. Przy instalacji **zaznacz „Add Python to PATH”**.
6. **ffmpeg** (do głosówek z Telegrama): `winget install ffmpeg`.
7. (Opcjonalnie) **Klucz API Claude**: console.anthropic.com, doładuj 5 \$, **ustaw limit wydatków**.



Tokenów i kluczy nigdy nikomu nie wysyłaj i nie wrzucaj do repo. Trzymamy je tylko w pliku `.env` na Twoim komputerze.

Etap 1 Jarvis MVP: tekst w Telegramie

1 wieczór

- Bot odbiera wiadomości, wysyła je do Ollamy (`qwen2.5:7b`) i odpisuje.
- Osobowość Jarvisa w system prompcie: lekko sarkastyczny brytyjski lokaj, zwraca się „sir” 😊
- 🇵🇱 **Wymuszony polski**: twarda instrukcja w system prompcie „odpowiadaj ZAWSZE po polsku”, bo Qwen lubi uciekać w angielski/chiński.
- ⚙️ **Większy kontekst**: Ollama domyślnie ma małe okno kontekstu, więc ustawiamy `num_ctx` na ~8192, żeby Jarvis nie gubił wątku.
- 🔒 **Whitelist**: bot odpowiada **tylko na Twoje Telegram ID**.
- Historia ograniczona do ostatnich ~10-15 wiadomości (krótszy kontekst to szybszy i mądrzejszy mały model).
- Komendy: `/start` , `/reset` (czyści rozmowę), `/model` (pokazuje aktualny mózg).

Etap 2 Pamięć

1 wieczór

- Baza **SQLite** z faktami o Tobie („lubi X”, „pracuje w Y”) i notatkami.
- Starsze rozmowy zamieniane na krótkie **podsumowanie**, bo mały model gorzej radzi sobie z długą historią.
- Do promptu wstrzykujemy tylko istotne fakty, nie całą bazę.

Etap 3 Głos

1-2 wieczory

- Wysyłasz głosówkę w Telegramie, ffmpeg i Whisper robią z niej tekst, a Jarvis odpowiada.
- Opcjonalnie odpowiedź głosówką przez **Piper TTS** (polskie głosy, binarki na Windowsa).
- `/głos on` / `/głos off` przełącza tryb odpowiedzi.
- ⚠️ Whisper i Ollama dzielą kartę graficzną. Na start Whisper `base` / `small` na **CPU**, żeby nie zabierał VRAM Qwenowi.

Etap 4 Narzędzia: Jarvis coś robi

po kawałku

Qwen 2.5 obsługuje **tool calling** w Ollamie, więc Jarvis sam zdecyduje, kiedy użyć narzędzia:

- 🕒 **Przypomnienia**: „przypomnij mi jutro o 9 o dentyście” (APScheduler i SQLite).
- ☀️ **Pogoda**: Open-Meteo (darmowe, bez klucza).
- 📝 **Notatki i listy**: zakupy, TODO.
- 💵 **Waluty**: darmowe API NBP.
- 📊 **Obliczenia, data i godzina**: liczymy w Pythonie, nie „na oko” modelu.
- 🔍 **Wyszukiwanie w necie**: przez darmowe DuckDuckGo albo przez Claude na żądanie.

💡 Mały model najlepiej działa z **niewielką liczbą prostych narzędzi** o jasnych opisach. Dodajemy je po jednym i testujemy.

Etap 5 Jarvis proaktywny

- 🌞 Codzienny **poranny brief** na Telegramie: pogoda i przypomnienia na dziś.
- Sam się odzywa, gdy zbliża się coś zaplanowanego.

Etap 6 Bajery na później

opcjonalnie

- ✨ **Tryb hybrydowy:** `/madry` przekazuje trudne pytanie do Claude.
- 🖥️ **Sterowanie Windowsem:** „odpal Spotify”, „wycisz kompa”, „zablokuj ekran”, „zrób screenshot”. Tylko z Twojego ID i z potwierdzeniem dla ryzykownych akcji.
- 📅 Google Calendar i Gmail.
- 🏠 Smart home (Home Assistant).
- 🎙️ Wake word „Jarvis” na mikrofonie w pokoju.
- 🔵 HUD z reaktorem łukowym w przeglądarce.
- 🔄 Test innych modeli: nowszy Qwen, polski Bielik.

🪟 Windows: rzeczy do ogarnięcia

- **Ollama** na Windowsie sama chodzi w tle (ikonka w trayu) i startuje z systemem. API jest pod `http://localhost:11434`.

Środowisko wirtualne (PowerShell)

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Jeśli PowerShell blokuje skrypt: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

- **Autostart Jarvisa:** Harmonogram zadań (Task Scheduler) albo skrót `.bat` w folderze `shell:startup`.
- **Działanie w tle:** uruchamianie przez `pythonw.exe`, bez okna konsoli.
- **Uśpienie komputera:** jak komp śpi, Jarvis też śpi. Do pracy 24/7 wyłącz usypianie w ustawieniach zasilania.

💰 Jak trzymać koszty (i obciążenie PC) nisko

1. Domyślnie **Ollama**, czyli 0 zł.
2. Claude **tylko na żądanie**, z **limitem wydatków** w konsoli.
3. Krótka historia i podsumowania, żeby mały model był szybszy i mądrzejszy.
4. **Whitelist**, czyli tylko Twoje Telegram ID.
5. Whisper na CPU, Ollama na GPU, żeby się nie gryzły.

Struktura projektu

```
Jarvis/
├── jarvis/
│   ├── bot.py           # Telegram (python-telegram-bot)
│   ├── brain/
│   │   ├── base.py      # wspólny interfejs mózgu
│   │   ├── ollama.py    # Ollama (qwen2.5:7b), domyślny
│   │   └── claude.py     # Claude (opcjonalny)
│   ├── memory.py        # SQLite + podsumowania
│   ├── voice.py         # Whisper + Piper
│   ├── config.py        # wczytywanie .env
│   └── tools/           # pogoda, przypomnienia, notatki, waluty...
├── data/                # baza SQLite, modele głosu (poza gitem)
├── .env                 # tokeny i ustawienia (NIGDY do gita!)
├── .env.example         # wzór pliku .env
├── .gitignore
├── requirements.txt
├── start_jarvis.bat     # odpalanie jednym kliknięciem
└── README.md
```

Przykładowy plik `.env`

```
TELEGRAM_TOKEN=...
ALLOWED_USER_ID=...
BRAIN=ollama
OLLAMA_MODEL=qwen2.5:7b
OLLAMA_URL=http://localhost:11434
# opcjonalnie:
ANTHROPIC_API_KEY=
```

Checklista postępów

- ☒ Ollama + qwen2.5:7b zainstalowane
- ☐ Etap 0: Przygotowanie (token bota, ID, Python, ffmpeg)
- ☐ Etap 1: MVP tekstowy w Telegramie
- ☐ Etap 2: Pamięć
- ☐ Etap 3: Głos
- ☐ Etap 4: Narzędzia
- ☐ Etap 5: Poranny brief
- ☐ Etap 6: Bajery

„Sir, plan gotowy. Czy mam zacząć od Etapu 0?”, J.A.R.V.I.S.